

Architettura dell'Agente AI "Spectra" — BioSpecInfo

Campo	Valore
Progetto	BioSpecInfo — componente Spectra (copilota AI agentico)
Autore	Samuele Pio Provenzano
Relatore tesi	Prof. Savino Longo — Università degli Studi di Bari Aldo Moro
Componente	<code>bsi-ai-hub.js</code> — 5.053 righe, nessuna dipendenza runtime
Tipo	Agente conversazionale multi-provider con esecuzione di strumenti lato client
Repository	<code>samupropiol-ship-it/BioSpecInfo-v11</code>
Versione documentata	Service Worker <code>bsi-v135</code>

1. Sintesi esecutiva

Spectra è un **agente** — non un semplice wrapper su un modello linguistico. La differenza è misurabile: il modello non risponde a memoria sui dati quantitativi, ma **invoca strumenti deterministici** che calcolano il risultato, e il ciclo di controllo gli permette di **concatenare fino a 10 chiamate** verificando ogni passaggio prima di rispondere.

L'architettura affronta tre problemi che distinguono un agente utilizzabile da una demo:

- Fondatezza (*grounding*)** — un dato chimico inventato può essere pericoloso. 32 strumenti coprono calcolo, banche dati pubbliche e i dataset interni dell'applicazione; il prompt di sistema vieta esplicitamente di citare valori numerici a memoria.
- Trasparenza** — il ragionamento del modello è mostrato in tempo reale e conservato accanto alla risposta, insieme all'elenco degli strumenti effettivamente invocati. L'operato dell'agente è verificabile a posteriori.
- Portabilità fra fornitori** — un unico registro di strumenti viene tradotto nei tre formati di *function calling* oggi in uso (OpenAI-compatibile, Anthropic, Gemini), così l'agente funziona identico su otto configurazioni di modello, incluse tre gratuite.

2. Architettura

2.1 Vista d'insieme



Non esiste un backend. Le chiavi API restano nel `localStorage` del browser e viaggiano solo verso il fornitore scelto: nessun dato transita da server di BioSpecInfo, che non esistono.

2.2 Ciclo agentico

Ad ogni giro il ciclo: invia la cronologia (ultimi 40 turni) più il prompt di sistema e lo schema dei 32 strumenti → riceve la risposta in *streaming* → se contiene chiamate a strumenti le esegue **tutte** → ricostruisce il turno assistente nel formato nativo del fornitore → ripete.

Tre condizioni di uscita: nessuna chiamata a strumenti (risposta finale), `MAX_ROUNDS = 10` esaurito, o interruzione dell'utente.

2.3 Registro strumenti neutrale rispetto al fornitore

Gli strumenti sono definiti una sola volta in JSON-Schema e tradotti a runtime:

Famiglia	Formato richiesto
Anthropic	<code>{name, description, input_schema}</code>
Gemini	<code>[{functionDeclarations: [...]}]</code>
OpenAI-compatibile	<code>{type:"function", function:{...}}</code>

Lo stesso vale per i messaggi: i turni con chiamate a strumenti vengono serializzati nella forma nativa di ciascun fornitore e conservati in un campo `_native`, così una conversazione resta

coerente anche cambiando modello.

2.4 Risoluzione del modello Gemini

Google ritira i modelli dall'endpoint `v1beta` senza preavviso: un nome scritto nell'URL smette di funzionare e restituisce `404 – models/... is not found`. Per questo `PROVIDERS.gemini.model` parte a `null` e viene deciso a runtime:

- 1. **ListModels** — si chiede all'API quali modelli quella chiave vede davvero. Ognuno riceve un punteggio: versione più recente, famiglia `flash`, con penalità per varianti sperimentali, `preview` e `-lite`; sono scartati i modelli che non espongono `streamGenerateContent` e quelli non conversazionali (embedding, immagini, audio nativo, Live API).
- 2. **Sonda dei candidati** — se `ListModels` non risponde (chiave limitata, rete, CORS), si prova una `GET` sui metadati di ogni nome noto: verifica reale, senza consumare quota di generazione.
- 3. **Primo candidato** — ultima riserva, per lasciar parlare l'errore vero della chiamata di generazione invece di inventarne uno qui.

La scelta è in cache per sette giorni, legata a un'impronta FNV-1a a 32 bit della chiave (chiavi diverse vedono cataloghi diversi; la chiave in chiaro non viene duplicata). Se il modello viene ritirato mentre è in cache, il `404` della chiamata di generazione invalida la cache, ririsolve e ritenta **una volta sola** — un flag impedisce il ciclo infinito quando anche il modello nuovo dà `404`.

3. I 32 strumenti

Area	Strumenti
Calcolo generale	calcola, risolvi_equazione, analisi_dati
Chimica generale	bilancia_equazione, stechiometria, massa_molecolare, converti_unita, costante_fisica
Chimica fisica	termodinamica, equilibrio_acido_base, cinetica, gas_e_soluzioni, elettrochimica
Struttura e spettri	spettroscopia, quantistica_e_spettroscopia, cristallografia
Scienze della vita	biochimica, farmacocinetica, valuta_druglikeness
Fisica	astrofisica, nucleare, statistica_inferenziale
Banche dati esterne	cerca_pubchem (NIH), cerca_letteratura (PubMed), ricerca web
Dati interni	cerca_nel_database (9 dataset), cerca_molecola
Controllo app	naviga_sezione (84 sezioni), apri_strumento (12 laboratori), stato_app
Memoria	ricorda, ricordi
Animazioni	apri_animazione (6 meccanismi di reazione)

3.1 Dataset interni esposti

297 reazioni di sintesi · 118 elementi · 67 amminoacidi · 143 farmaci · 63 patologie · 39 strategie retrosintetiche · 36 interazioni farmacologiche · 29 vie metaboliche · 29 potenziali redox.

Sono i dati curati dall'autore per l'applicazione: l'agente li consulta come fonte primaria e ha istruzione di **segnalare le discrepanze** fra database e propria memoria invece di appianarle silenziosamente.

4. Decisioni ingegneristiche rilevanti

Questa sezione documenta le scelte non ovvie e il motivo per cui sono state prese. È la parte del documento che descrive il ragionamento progettuale.

4.1 Motore di calcolo senza `eval()`

Problema. Servire calcoli arbitrari di chimica fisica richiede un valutatore di espressioni. `eval()` lo risolverebbe in poche righe.

Perché è stato escluso. L'agente legge contenuti esterni non fidati (risultati PubChem, abstract PubMed, pagine web). Un'iniezione in quei contenuti potrebbe indurre il modello a emettere un'espressione ostile; con `eval()` quella stringa verrebbe eseguita nel contesto della pagina, **dove risiede la chiave API dell'utente**.

Soluzione. Parser a discesa ricorsiva con insieme *chiuso* di 27 funzioni e costanti. Non ha accesso all'ambito globale: può solo fare aritmetica. Verificato con 10 tentativi di evasione (`window.localStorage`, `constructor`, `this`, `fetch(...)`, chiamate a funzioni non in whitelist), tutti respinti, anche eseguendo il test in un browser reale.

4.2 Conservazione dei blocchi di ragionamento

Sui modelli con ragionamento adattivo i blocchi `thinking` fanno parte del turno assistente e devono essere **rimandati indietro identici, firma crittografica inclusa**, nel giro successivo di *tool use*. Scartarli fa rifiutare la richiesta.

L'implementazione accumula `thinking_delta` e `signature_delta` dallo stream e li reinserisce **in testa** al turno ricostruito. Verificato in test d'integrazione: firma preservata, ordine corretto.

4.3 Ripresa dei turni sospesi (`pause_turn`)

La ricerca web è uno strumento eseguito sui server del fornitore. Quando il suo ciclo interno esaurisce le iterazioni disponibili, la risposta termina con `stop_reason`: `"pause_turn"`.

Il turno va ripreso **rimandando indietro il turno assistente così com'è, senza aggiungere alcun messaggio utente**: è il server a riconoscere il blocco `server_tool_use` in coda e a riprendere da lì. Aggiungere un "continua" spezzerebbe il meccanismo.

4.4 Configurazione per modello, non globale

I quattro livelli Claude non accettano gli stessi parametri:

Modello	effort	Ricerca web	Note
Fable 5.1	xhigh	sì (8)	fallback su rifiuto; ragionamento sempre attivo
Opus 5	high	sì (6)	predefinito
Sonnet 5	high	sì (5)	
Haiku 4.5	non supportato	non supportata	inviarli produce HTTP 400

Su tutti i modelli recenti `temperature`, `top_p` e `budget_tokens` sono stati rimossi dall'API e la loro presenza causa un errore: nessuno dei quattro li invia. Ogni parametro è quindi condizionato al fornitore, non impostato globalmente.

4.5 Ingresso multimodale e lettura dei materiali

L'agente accetta immagini, PDF e file di testo, anche molti insieme, trattati come pagine consecutive di un unico documento. Tre vincoli hanno guidato l'implementazione:

- **Le immagini vengono ridimensionate a 2000 px** e ricomprese in JPEG prima dell'invio. Una foto da telefono passa da circa 1 MB a 440 KB: senza questo passaggio una singola immagine avvicinerebbe il tetto di richiesta e moltiplicherebbe il costo in token. 2000 px sono il minimo per leggere una grafia minuta — a 1600 px la scrittura piccola diventava illeggibile.
- **Il tetto di pagine dei PDF dipende dal modello**, non è una costante: 600 pagine per i modelli con finestra da 1M, 100 per quelli da 200K. Il conteggio avviene leggendo gli oggetti `/Type` `/Page` nei byte del file, senza librerie.
- **Nella cronologia va una miniatura da 160 px**, non l'immagine inviata. Salvare quella grande (440 KB l'una) riempiva `localStorage` in una decina di foto, e a quota piena ogni scrittura successiva fallisce in silenzio.

Sei azioni rapide trasformano il materiale in un prodotto di studio (riassunto, mappa concettuale, schema, flashcard, domande d'esame, trascrizione) e due lo traducono. Per le trascrizioni l'agente ha istruzione di scrivere `[illeggibile]` invece di indovinare: un termine chimico inventato è peggio di una lacuna dichiarata.

4.6 Renderer di grafi proprio, senza CDN

Le mappe concettuali sono prodotte dal modello come diagrammi Mermaid e disegnate da un renderer scritto nell'applicazione (~190 righe): livelli per cammino più lungo dalle radici con interruzione dei cicli, riordino per baricentro dei predecessori su tre passate per ridurre gli incroci, output SVG con archi di Bézier.

La scelta di non caricare Mermaid da CDN nasce dalla natura offline-first dell'app: una mappa deve potersi disegnare anche senza rete. Le sintassi non coperte vengono riconosciute e lasciate come blocco di codice leggibile, invece di essere disegnate male.

4.7 Comando vocale con parola di attivazione

Ascolto continuo attivabile dall'utente: la frase «Hey Spectra» seguita dal comando lo invia automaticamente. Due dettagli non ovvi:

- il riconoscimento vocale del browser **termina da solo** dopo qualche secondo di silenzio e va riavviato, altrimenti l'ascolto muore alla prima pausa;
- vengono esaminate **tutte le alternative di trascrizione**, non solo la prima, e sono accettate le varianti fonetiche più comuni: il riconoscimento italiano rende "Spectra" in molti modi diversi.

Il rifiuto del permesso al microfono interrompe il ciclo invece di ritentare all'infinito. Il termine "spettroscopia", frequentissimo in questo dominio, è stato verificato come non attivante.

4.8 Guardia sui risultati non finiti

Un audit sistematico ha rivelato che, con input degeneri legittimi (lunghezza d'onda nulla, volume nullo, emivita nulla, transizione fra livelli identici), otto risolutori restituivano `ok: true` con `Infinity` o `NaN` fra i campi.

È il guasto più insidioso in questo contesto: un'eccezione si nota, un valore formalmente valido ma privo di senso viene riportato all'utente come corretto. La correzione è un controllo unico nel punto in cui transitano tutti i risultati: se un campo numerico non è finito, il risultato diventa un errore esplicito che nomina i campi e indica la causa probabile. Vale anche per gli strumenti aggiunti in futuro.

4.9 Degradazione controllata

- **Modelli senza *function calling* affidabile** (alcuni gratuiti): al primo errore riconducibile agli strumenti, la richiesta viene ripetuta una volta in modalità solo testo, invece di fallire.
- **Timeout di inattività (45 s)**, azzerato ad ogni byte ricevuto: una risposta lunga non viene troncata, ma una connessione morta viene segnalata subito con un messaggio esplicito.
- **Quota di `localStorage`**: lo storico è limitato (30 conversazioni, 100 messaggi) con potatura progressiva. Senza questo limite il superamento della quota faceva fallire *in silenzio* ogni scrittura successiva, **compreso il salvataggio della chiave API**.
- **Rifiuto dei classificatori** (`stop_reason: "refusal"`, HTTP 200): rilevato e comunicato, invece di presentarsi come risposta vuota.

5. Verifica

La verifica è stata condotta su tre livelli, con esiti registrati.

5.1 Correttezza numerica — confronto con valori di riferimento

Ambito	Caso	Atteso	Ottenuto
Bilanciamento	$\text{KMnO}_4 + \text{HCl} \rightarrow \dots$	2:16:2:2:8:5	✓ esatto
Massa molecolare	$\text{K}_4[\text{Fe}(\text{CN})_6]$ (annidata)	368,35	368,345
Massa molecolare	$\text{CuSO}_4 \cdot 5\text{H}_2\text{O}$ (idrato)	249,68	249,677
Equilibrio	acido acetico 0,1 M, K_a $1,8 \cdot 10^{-5}$	pH 2,874	2,875
Termodinamica	ΔG ($\text{N}_2\text{O}_4/\text{NO}_2$) a 298 K	+4,79 kJ/mol	✓
Cinetica	E_a da $k(300\text{ K})$ e $k(320\text{ K})$	55,3 kJ/mol	55,33
Gas reali	van der Waals CO_2	scostamento -10 %	-10,13 %
Spettroscopia	M+2 del bromo	97,3 %	✓
Biochimica	glicilglicina	132,12 Da	✓
Farmacocinetica	$t_{1/2}$ da V_d 50 L, Cl 5 L/h	6,93 h	✓
Astrofisica	picco di emissione solare (Wien)	501,5 nm	501,52
Astrofisica	velocità di fuga terrestre	11,19 km/s	11,186
Nucleare	energia di legame ^{56}Fe	8,79 MeV/nucleone	8,790
Statistica	t critico, 95 %, $df = 4$	2,7764	✓
Cristallografia	densità del rame (fcc)	8,96 g/cm ³	8,935

I p-value non sono tabulati: sono calcolati con la funzione beta incompleta regolarizzata (frazione continua di Lentz) e la gamma incompleta.

5.2 Robustezza — casi che devono fallire

Verificato che vengano **respinti**: equazioni non bilanciabili, formule con parentesi non bilanciate o elementi inesistenti, specie assenti dall'equazione, tipi di database non validi, e i 10 tentativi di iniezione nel motore di calcolo.

5.3 Integrazione — browser reale

Test automatizzati con Chromium *headless* su tutte le 14 pagine dell'applicazione: nessun errore JavaScript, nessuna risorsa mancante. Percorse una per una le 84 sezioni di [index.html](#) e i 18 tab della sezione Astrochimica. Ispezionato il corpo delle richieste per i quattro livelli Claude, e simulato un ciclo completo con ricerca web, sospensione e ripresa del turno.

6. Riepilogo quantitativo

Metrica	Valore
Righe del componente	5.053
Strumenti	32
Configurazioni di modello	8 (di cui 3 gratuite)
Aree scientifiche coperte	13
Record nei dataset interni esposti	oltre 800
Giri massimi del ciclo agentico	10
Cronologia inviata al modello	40 turni
Dipendenze runtime	nessuna

7. Limiti dichiarati

Documentati per onestà tecnica; sono scelte consapevoli, non omissioni.

- **La chiave API risiede nel browser.** È l'unica opzione senza backend: la chiave non lascia il dispositivo, ma è accessibile a chi ha accesso a quel browser. Una chiave condivisa richiederebbe un proxy server con quota, fuori dal perimetro di un'applicazione statica.
- **Gli strumenti di rete dipendono dalla disponibilità dei servizi.** PubChem e PubMed sono interrogati direttamente dal browser; in caso di irraggiungibilità lo strumento restituisce un errore esplicito e l'agente ha istruzione di dichiararlo, non di sostituirlo con una stima.
- **I risolutori applicano modelli semplificati** dove la letteratura ne prevede di più raffinati (per esempio Woodward-Fieser per l'UV, o le rese in ATP con rapporti P/O medi). Ogni strumento dichiara il metodo usato nel proprio risultato.
- **La verifica è funzionale e numerica**, non formale: non esiste prova di correttezza dei risolutori, ma un insieme di casi di riferimento tratti dalla letteratura didattica.

Documento redatto a corredo del dossier tecnico BioSpecInfo. Vedi anche: [01-Software-Architecture-Document.md](#), [02-Verification-Validation-Report.md](#), [03-Security-Privacy-Compliance.md](#).